

--	--	--	--	--	--	--	--	--	--

Date: 18/03/2026
Duration: 1 hour 45

CONTINUOUS ASSESSMENT 1
II YEAR – SEMESTER 4
CSE23AE204 - PCP III
(B.Tech. E01,E02,E03,E05,E06, E61)

SET C

Question 1: Problem Statement

You are given a JSON object containing a list of items and the capacity of a knapsack.

```
{
  "capacity": W,
  "items": [[value, weight], ...]
}
```

Each item has:

- value → value of the item
- weight → weight of the item

Unlike the 0/1 knapsack problem, **you are allowed to take fractions of an item.**

Your task is to maximize the **total value that can be placed in the knapsack.**

Strategy:

Select items based on the **highest value-to-weight ratio first.**

Output Format

Return the **maximum value that can be obtained.**

Example 1

Input

```
{
  "capacity": 50,
  "items": [[60,10],[100,20],[120,30]]
}
```

Output

240.0

Explanation

Compute value/weight ratio:

Take items greedily:

- Take item1 → value = 60
- Take item2 → value = 160
- Take **20 weight of item3** → value = 80

Total value = **240**

Question 2: Problem Statement

An array of non-negative integers represents the **maximum jump length** that can be made from each index.

Starting at the **first index**, determine the **minimum number of jumps required to reach the last index.**

From position i , a jump can reach any position in the range:

$i + 1$ to $i + \text{nums}[i]$

It is guaranteed that the last index is reachable.

Input Format

`{"nums": [a1, a2, a3, ...]}`

Where:

- `nums[i]` represents the **maximum jump length** from index i .

Output Format

Return:

`minimum_jumps`

Where:

- `minimum_jumps` → minimum number of jumps needed to reach the last index.

Example 1

Input

`{"nums": [2,3,1,1,4]}`

Output

2

Explanation

Optimal path:

index 0 → jump to index 1

index 1 → jump to index 4

Minimum jumps = 2

Question 3: Problem Statement

You are given an array of **N integers**, where each element represents the **maximum number of steps you can jump forward** from that position.

You start at the **first index (index 0)** of the array.

From any position i , you can jump to any position between:

$i + 1$ to $i + \text{arr}[i]$

Your task is to determine whether it is possible to **reach the last index of the array** starting from the first index.

Return:

- **true** if the last index can be reached
- **false** otherwise

Example 1

Input

5

2 3 1 1 4

Output

true

Explanation

Possible jumps:

index 0 → jump 1 step → index 1

index 1 → jump 3 steps → index 4

The last index is reachable.